KU LEUVEN

FACULTEIT INGENIEURSWETENSCHAPPEN

# Getting Acquainted with GPU Programming
# Lab 1
## Geavanceerde computerarchitectuur

Wouter Fouquet
Florian Vandenbogaerde
Master elektronica ICT
Academiejaar 2024-2025

CONTENTS

**Exercise** **1**
0.0.1 Problem . . . . . 1

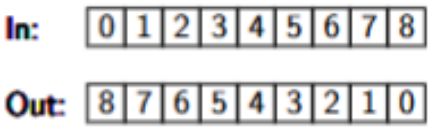
**1 Methodology** **2**
1.0.1 Objective . . . . . 2
1.0.2 Results . . . . . 2

**Conslusion** **4**

EXERCISE

### 0.0.1 Problem

We have to flip an input array to obtain an output array, see Figure 1. To do so, we use multiple blocks with multiple threads. Additionally, we perform measurements to compare different configurations and determine the most optimal setup. By varying parameters such as the number of threads per block and blocks per grid, we analyze the efficiency of both CPU and GPU execution, identifying the best balance between performance and resource utilization.




Figuur 1: Swap

### 1.0.1 Objective

In modern computing, optimizing performance depends on choosing the right hardware for the task. CPUs are designed for sequential execution and handle small datasets efficiently due to their fast cache access and low overhead. GPUs, however, are built for massive parallelism, making them better suited for processing large amounts of data simultaneously. The efficiency of GPU execution depends on factors such as thread distribution and memory management.

In this lab, we flip an array using both CPU and GPU implementations. By varying the number of threads per block and blocks per grid, we measure execution times and analyze performance differences. This allows us to determine the most efficient configuration and understand at what point the GPU outperforms the CPU.

### 1.0.2 Results

Table 1.1 shows the results of all the measurements we did during this lab session.

| **Array Size** | **Threads per Block** | **Blocks per Grid** | **Time (GPU) [$\mu$s]** | **Time (CPU) [$\mu$s]** |
|---|---|---|---|---|
| 9 | 3 | 3 | 296 | 0.2 |
| 9 | 1 | 9 | 583 | 0.3 |
| 9 | 9 | 1 | 219 | 0.1 |
| 100 | 10 | 10 | 358 | 0.4 |
| 100 | 1 | 100 | 216 | 0.4 |
| 100 | 100 | 1 | 264 | 0.4 |
| 1000 | 10 | 100 | 377 | 2.4 |
| 1000 | 1 | 1000 | 260 | 2.4 |
| 1000 | 1000 | 1 | 262 | 2.4 |
| 10000 | 100 | 100 | 214 | 23 |
| 10000 | 10 | 1000 | 41 | 41 |
| 10000 | 1000 | 10 | 190 | 26 |
| 100000 | 1000 | 100 | 319 | 291 |
| 100000 | 100 | 1000 | 302 | 289 |

Tabel 1.1: Vergelijking van uitvoeringstijden voor array-flipping op GPU en CPU

The CPU is optimized for sequential processing and efficiently utilizes fast caches and advanced instruction sets. As a result, it can process small datasets quickly without additional overhead. The GPU, on the other hand, excels at parallel computation, which is beneficial for large datasets. However, the initial overhead, such as launching kernels and transferring data to GPU memory, makes it less efficient for smaller arrays.

When determining the optimal GPU configuration, the balance between the number of threads per block and the number of blocks per grid is crucial. Since GPUs operate with warps of 32 threads, it is essential to use a multiple of this number to fully utilize the computing cores. Typically, the optimal number of threads per block ranges between 128 and 1024, depending on the array size and available hardware. However, we cannot exceed 1024 threads per block, as this is the maximum allowed. If we attempt to go beyond this limit, our output array will contain "0"(unwritten) values beyond 1024. While it is possible to reassign these open positions to existing threads by adjusting the formula and position calculations, for simplicity, we assume that there are always enough threads available to fully process the array [1].

The choice of how to distribute threads and blocks significantly affects performance:

- Many blocks with few threads per block leads to excessive overhead, as the GPU must manage a large number of blocks, increasing scheduling complexity.
- Few blocks with many threads per block can result in sequential delays if there aren't enough blocks to utilize all multiprocessors efficiently.
- A balanced compromise between both strategies mitigates the drawbacks, ensuring a good distribution of workload while keeping scheduling overhead manageable.

Our measurements show that, within our tested range, the GPU is never faster than the CPU. This is likely because we are unable to make the arrays large enough to fully exploit the GPU's parallel processing power. However, based on theoretical performance scaling, we assume that for significantly larger arrays (e.g., one million elements or more), the GPU would become faster than the CPU. In such cases, the high degree of parallelism would compensate for the initial overhead, leading to better performance compared to sequential CPU execution.

Thus, the choice between CPU and GPU strongly depends on the scale of the task and the correct utilization of computational resources. While the CPU remains dominant for smaller arrays, the GPU has the potential to significantly outperform it when handling sufficiently large datasets.

## CONCLUSION

Our results show that, within the tested range, the CPU consistently outperforms the GPU. The main reason is that our arrays are not large enough to fully utilize the GPU's parallel processing power. Additionally, choosing the right balance between threads per block and blocks per grid is essential to minimize overhead and maximize efficiency. While many small blocks introduce excessive scheduling overhead, too few blocks can lead to underutilization of GPU resources. Based on theoretical scaling, we expect that for significantly larger arrays (e.g., one million elements or more), the GPU would eventually surpass the CPU in efficiency. This highlights the importance of selecting the right hardware depending on the dataset size and computational requirements.

# Bibliografie

[1] Wikipedia. *Thread block* (*CUDA programming*). 2016. URL: https://en.wikipedia.org/wiki/Thread_block_%28CUDA_programming%29? (bezocht op 14-02-2025).

```cpp
#include <stdio.h>
#include <cuda_runtime.h>
#include <chrono>

using namespace std;
using namespace std::chrono;

// GPU function to reverse Arrays
__global__ void reverseArrayGPU(int *arrayInGPU, int *arrayOutGPU, int sizeArray) {
    int index = threadIdx.x + blockIdx.x * blockDim.x;
    int swapIndex = sizeArray - index - 1;

    if(index < sizeArray){
        arrayOutGPU[index] = arrayInGPU[swapIndex];
    }
}

// CPU function to reverse Arrays
void reverseArrayCPU(int *arrayIn, int *arrayOut, int sizeArray) {
    for (int i = 0; i < sizeArray; i++) {
        arrayOut[i] = arrayIn[sizeArray - i - 1];
    }
}



int main(){
    const int sizeArray = 100000;
    const int threadsPerBlock = 100;
    const int blocksPerGrid = 1000;

    // define array
    int arrayInCPU[sizeArray];
    for (size_t i = 0; i < sizeArray; i++){
        arrayInCPU[i] = i;
        //printf("%d ", arrayInCPU[i]);
    }
    printf("\n");

    int arrayOutCPU[sizeArray];

    //Start timer
    auto startCpu = high_resolution_clock::now();

    //Swap CPU
    reverseArrayCPU(arrayInCPU, arrayOutCPU, sizeArray);

    //Stop Timer
    auto stopCpu = high_resolution_clock::now();
    auto durationCpu = duration_cast<nanoseconds>(stopCpu - startCpu);
    printf("CPU-time: %lld ns\n", durationCpu.count());

    //check if swap is correct
    /*
    for (int i = 99950; i < sizeArray; i++) {
        printf("%d ", arrayOutCPU[i]);
    }
    printf("\n");
    */
```

```
60
61
62      //GPU PART
63
64
65      // Memory allocation
66      int* arrayInGPU;
67      int* arrayOutGPU;
68      size_t totalSizeGPU = sizeArray * sizeof(int);
69      cudaMalloc((void**)&arrayInGPU, totalSizeGPU);
70      cudaMalloc((void**)&arrayOutGPU, totalSizeGPU);
71
72      //copy to GPU memory
73      cudaMemcpy(arrayInGPU, arrayInCPU, totalSizeGPU, cudaMemcpyHostToDevice
            );
74
75      // Start Timing GPU
76      cudaEvent_t start_cuda, stop_cuda;
77      cudaEventCreate(&start_cuda);
78      cudaEventCreate(&stop_cuda);
79      cudaEventRecord(start_cuda);
80
81      reverseArrayGPU<<<blocksPerGrid, threadsPerBlock>>>(arrayInGPU,
            arrayOutGPU, sizeArray);
82      cudaMemcpy(arrayOutCPU, arrayOutGPU, totalSizeGPU,
            cudaMemcpyDeviceToHost);
83
84      cudaDeviceSynchronize();
85      cudaEventRecord(stop_cuda);
86      cudaEventSynchronize(stop_cuda);
87      float msCuda;
88      cudaEventElapsedTime(&msCuda, start_cuda, stop_cuda);
89
90      //check if swap is correct
91      /*
92      for (int i = 99950; i < sizeArray; i++) {
93          printf("%d ", arrayOutCPU[i]);
94      }
95      printf("\n");
96      */
97
98      printf("GPU-time:␣%f␣ms\n", msCuda);
99
100     // Free memory
101     cudaFree(arrayInGPU);
102     cudaFree(arrayOutGPU);
103
104     return 0;
105 }
```